

KUBERNETS

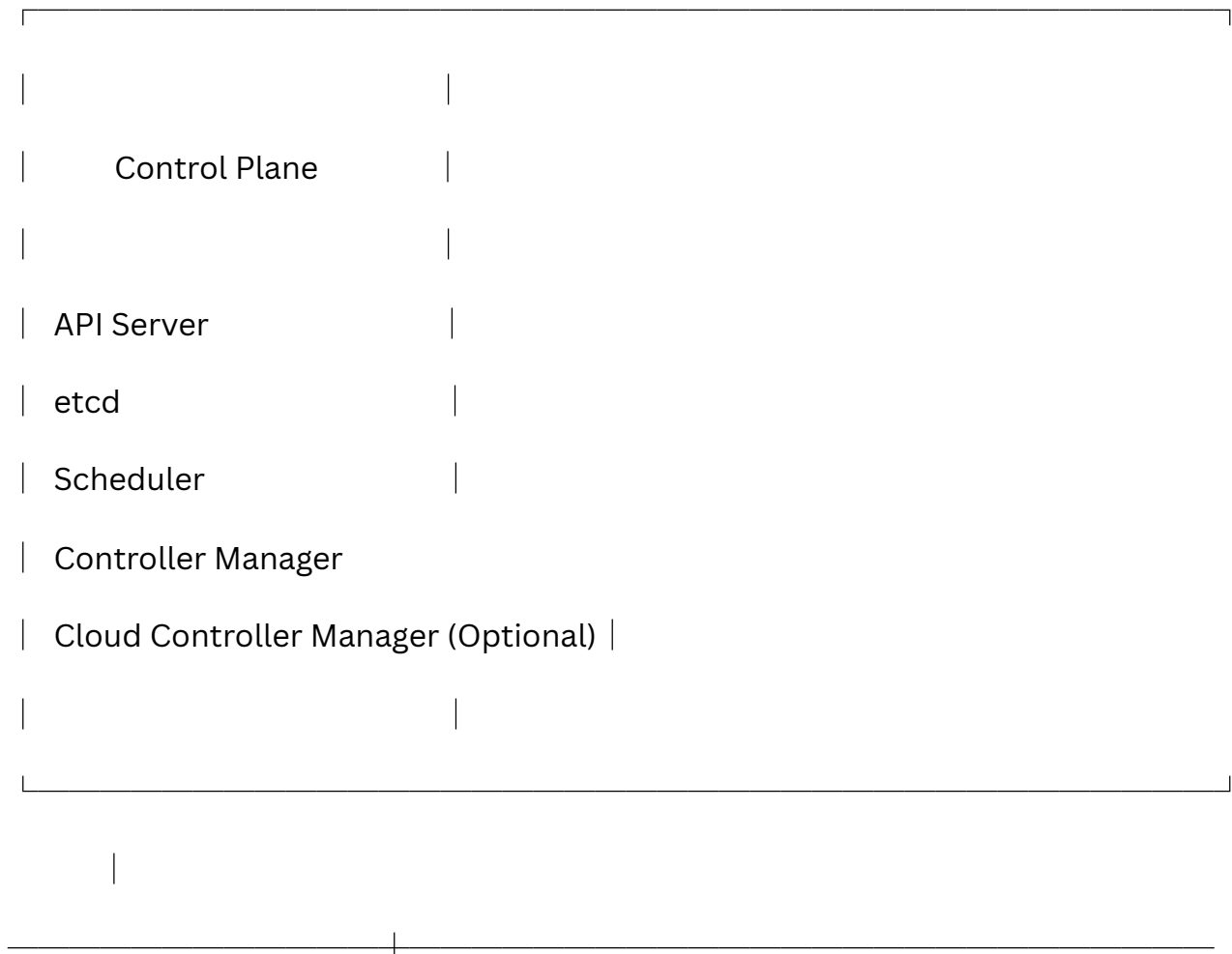
Kubernetes is an **open-source container orchestration platform** that automates the deployment, scaling, management, and monitoring of containerized applications.

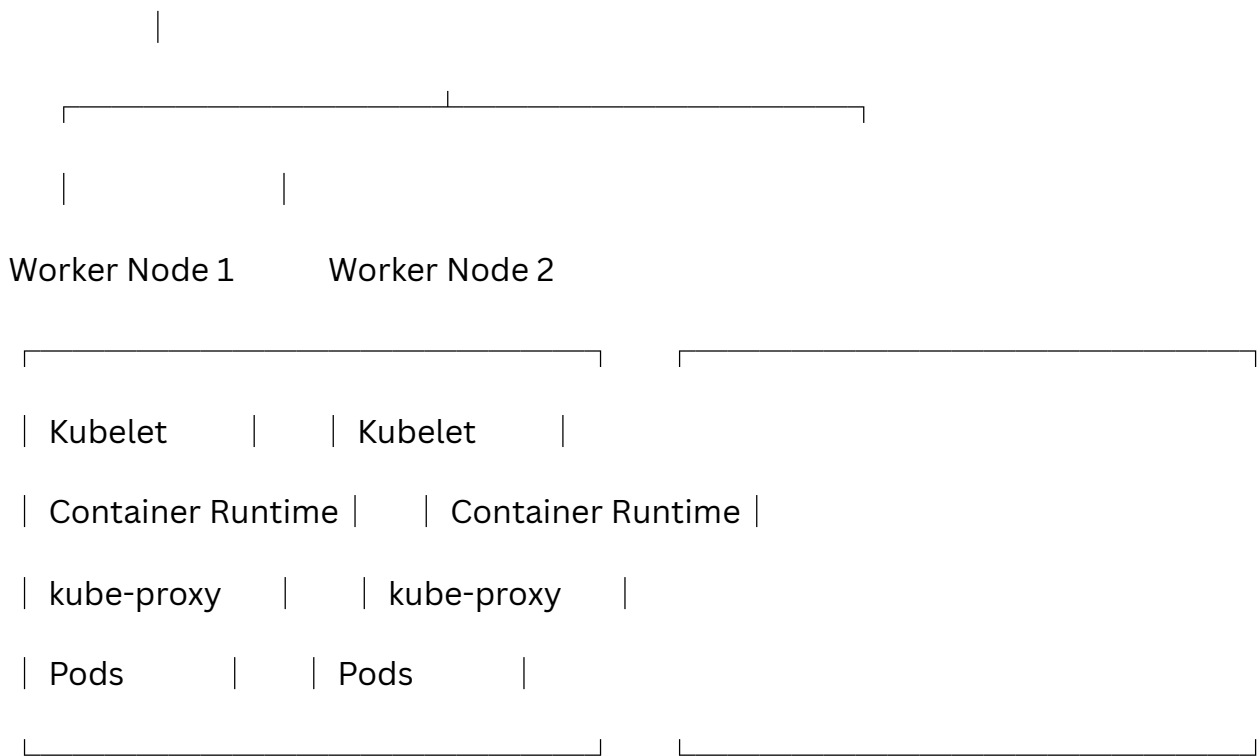
Kubernetes Architecture

Kubernetes architecture is divided into **two main parts**:

1. **Control Plane (Master Node)** – The brain of Kubernetes.
2. **Worker Node** – The machines where your applications actually run.

Kubernetes Cluster





1. Kubernetes Cluster

A **Kubernetes Cluster** is a collection of machines (nodes) working together to run your applications.

It contains:

- **Control Plane**
- **One or more Worker Nodes**

Think of it as a company.

- Control Plane = Manager
- Worker Nodes = Employees

2. Control Plane (Brain of Kubernetes)

The **Control Plane** makes all the decisions.

It:

- Receives requests
- Decides where Pods should run
- Monitors the cluster
- Maintains the desired state

Imagine a company manager assigning work to employees.

The Control Plane contains five main components.

3. API Server

The **API Server** is the entry point of Kubernetes.

Everything communicates with Kubernetes through the API Server.

Responsibilities

- Accepts requests
- Validates requests
- Stores information in etcd
- Returns responses
- Communicates with all components

Example

```
kubectl apply -f deployment.yaml
```

↓

API Server receives request

↓

Validates YAML

↓

Stores data in etcd

↓

Notifies Scheduler

4. etcd

etcd is the database of Kubernetes.

It stores the entire cluster state.

Examples:

- Pods
- Deployments
- Secrets
- ConfigMaps
- Nodes
- Services

Without etcd, Kubernetes cannot remember anything.

Think of etcd as the cluster's memory.

5. Scheduler

The **Scheduler** decides **which Worker Node should run a new Pod**.

It checks:

- Available CPU
- Available RAM
- Node health
- Taints and tolerations
- Node affinity
- Resource requests

It **does not create Pods**. It only selects the best node.

6. Controller Manager

The **Controller Manager** continuously checks whether the actual cluster matches the desired state.

Example:

Desired Pods = 3

Actual Pods = 2

The Controller Manager notices the difference and creates one more Pod to reach the desired state.

It includes controllers such as:

- Deployment Controller
- ReplicaSet Controller
- Node Controller
- Job Controller
- Namespace Controller

7. Cloud Controller Manager (Optional)

This component is used when Kubernetes runs on cloud providers.

It manages cloud resources like:

- Load Balancers
- Cloud storage
- Cloud networking
- Cloud instances

On local clusters like Minikube, it is usually not used.

8. Worker Node

A **Worker Node** is where your application actually runs.

Each Worker Node contains:

- Kubelet
- Container Runtime
- kube-proxy
- Pods

9. Kubelet

Kubelet is the agent running on every Worker Node.

Responsibilities:

- Talks to the API Server
- Creates Pods
- Starts containers
- Monitors Pods
- Reports Pod status back to the Control Plane

Think of Kubelet as the supervisor of the Worker Node.

10. Container Runtime

The **Container Runtime** runs containers.

Examples include:

- containerd
- CRI-O

Responsibilities:

- Pull container images
- Start containers
- Stop containers
- Delete containers

11. kube-proxy

kube-proxy handles networking.

Responsibilities:

- Routes traffic to Pods
- Implements Kubernetes Services
- Load balances traffic between Pods

Example:

User

|



Service

|

├── Pod 1

├── Pod 2

└── Pod 3

kube-proxy distributes requests among the available Pods.

12. Pod

A **Pod** is the smallest deployable unit in Kubernetes.

A Pod contains:

- One or more containers
- Shared IP address
- Shared storage (if configured)
- Shared network namespace

Usually, one Pod runs one main application container.

Complete Workflow

Developer

|

kubectl apply



API Server



etcd stores desired state



Scheduler selects Worker Node



Kubelet receives instruction



Container Runtime pulls image



Pod starts



kube-proxy exposes networking



Application is running

Easy way to remember

Component	Simple role
Control Plane	Brain that manages the cluster
API Server	Entry point for all requests
etcd	Database that stores cluster state
Scheduler	Chooses the best Worker Node for a Pod
Controller Manager	Keeps the actual state matching the desired state
Cloud Controller Manager	Manages cloud-specific resources
Worker Node	Runs your applications
Kubelet	Agent that manages Pods on a node
Container Runtime	Runs containers
kube-proxy	Handles networking and load balancing
Pod	Smallest deployable unit that contains containers

Deployment in Kubernetes

Definition

A Deployment is a Kubernetes resource that manages Pods and ensures the desired number of application instances are always running.

Short definition (Interview)

A Deployment is a Kubernetes object that automates the creation, updating, scaling, and self-healing of Pods.

Simple Explanation (Thanglish)

Imagine you own a restaurant.

You tell your manager:

"Always keep 3 chefs working in the kitchen."

Now:

- If one chef leaves, the manager hires another.
- If you want 5 chefs, the manager hires 2 more.
- If you update the menu, the manager trains the chefs one by one without closing the restaurant.

Here:

- You → Developer
- Manager → Deployment
- Chefs → Pods

So, Deployment's job is to make sure the required number of Pods are always running.

Why do we use Deployment?

Without a Deployment:

- You create Pods manually.
- If a Pod crashes, you must recreate it yourself.
- Updating the application is difficult.

With a Deployment:

- Pods are created automatically.
- Crashed Pods are recreated automatically.
- Scaling is easy.
- Updates happen with minimal downtime.
- Rollbacks are supported if an update fails.

Architecture

Deployment

|



ReplicaSet

|



Pods

What each component does

- Deployment → Manages the application's lifecycle.
- ReplicaSet → Ensures the desired number of Pods are running.
- Pods → Run your application containers.

Example

Suppose you run:

```
kubectl create deployment nginx --image=nginx --replicas=3
```

Kubernetes creates:

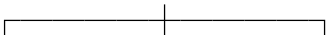
Deployment

|



ReplicaSet

|



Pod1 Pod2 Pod3

Now imagine:

Pod2 crashes ❌

ReplicaSet notices only 2 Pods are running instead of the desired 3.

It automatically creates a new Pod:

Pod1

Pod3

New Pod2

You don't need to do anything manually.

Main Responsibilities of a Deployment

- Create Pods
- Scale Pods up or down
- Perform rolling updates
- Roll back to a previous version
- Ensure the desired number of Pods are always running

Real-world Example

Think about a shopping website.

Suppose 3 Pods are serving users:

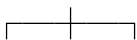
Users

|



Service

|



Pod Pod Pod

During a festival sale, traffic increases.

You change:

replicas: 10

Deployment instructs the ReplicaSet to create 10 Pods.

When traffic decreases:

replicas: 3

Deployment removes the extra Pods while keeping the application available.

Easy way to remember

Component	Responsibility
Deployment	Manages application updates, scaling, and Pod lifecycle
ReplicaSet	Maintains the desired number of Pods
Pod	Runs the application container